

Reviewer Pack — Finite Plane Line Count Bounds Tseitin Resolution Size

Entry 595a1117d469 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 02:26:42 UTC

Finite Plane Line Count Bounds Tseitin Resolution Size

Entry ID: 595a1117d469 Verdict: INCONCLUSIVE Recorded: 2026-05-08 02:26:42 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry Field B (complexity object): Resolution Proof Size

Statement:

For a Tseitin formula derived from a finite projective plane of order $q \geq 2$, the minimal resolution proof size is $\Theta(q^3)$. The number of lines in the plane is $q^2 + q + 1$, and the conjecture asserts that the resolution size scales proportionally to q^3 for all valid q .

Rationale (proposer’s reasoning):

Finite geometry provides structured incidence graphs with inherent combinatorial rigidity. Tseitin formulas on such graphs inherit this rigidity, forcing resolution proofs to traverse a super-linear number of clauses. The cubic scaling arises from the interplay between the plane’s line count and the expansion properties of its incidence graph.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 60f7b291c5a27f05

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - finite projective plane Tseitin formula resolution proof size - finite geometry Tseitin formula resolution complexity - Tseitin formula resolution asymptotic complexity finite geometry

Top relevant hits considered: - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2111.12700v2>] Universality in long-distance geometry and quantum complexity - [<http://arxiv.org/abs/0711.0446v1>] Asymptotic Properties of Hilbert Geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def generate_primes(n):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, m):
            factor = A[j][i] / A[i][i]
```

```

        for k in range(n):
            A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
x = [0] * n
for i in range(m - 1, -1, -1):
    x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
return x

def dpll(clauses, assignment=None):
    if assignment is None:
        assignment = {}
    if not clauses:
        return True
    unit_clauses = [c[0] for c in clauses if len(c) == 1]
    pure_symbols = {}
    for clause in clauses:
        for literal in clause:
            symbol = abs(literal)
            if symbol not in pure_symbols:
                count = sum(1 for c in clauses if literal in c)
                pure_symbols[symbol] = 'positive' if count > len(clauses) - count else 'negative'
    unit_clauses.extend([symbol for symbol, polarity in pure_symbols.items() if polarity == 'positive'])
    unit_clauses.extend([-symbol for symbol, polarity in pure_symbols.items() if polarity == 'negative'])
    unit_clauses = list(set(unit_clauses))
    for literal in unit_clauses:
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll([c for c in clauses if not any(l in c or -l in c for l in new_assignment)], new_assignment):
            return True
        new_assignment[literal] = False
        if dpll([c for c in clauses if not any(l in c or -l in c for l in new_assignment)], new_assignment):
            return True
    return False

def generate_tseitin_formula(q):
    n = q * (q + 1) + 1
    variables = list(range(1, n + 1))
    clauses = []
    incidence_matrix = [[0] * n for _ in range(n)]
    for i in range(q):
        for j in range(q):
            line_index = i * q + j + 1
            point_index = (i * q + j) % (q + 1) + 1
            incidence_matrix[line_index][point_index] = 1
            clauses.append([line_index, -point_index])
            clauses.append([-line_index, point_index])
    for i in range(q):
        line_index = i * q + q + 1
        point_index = (i * q) % (q + 1) + 1
        incidence_matrix[line_index][point_index] = 1
        clauses.append([line_index, -point_index])
        clauses.append([-line_index, point_index])
    for i in range(q):
        line_index = i * q + q + 2
        point_index = (i * q + 1) % (q + 1) + 1
        incidence_matrix[line_index][point_index] = 1

```

```

    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 3
    point_index = (i * q + q) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 4
    point_index = (i * q + q + 1) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 5
    point_index = (i * q + q + 2) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 6
    point_index = (i * q + q + 3) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 7
    point_index = (i * q + q + 4) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 8
    point_index = (i * q + q + 5) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 9
    point_index = (i * q + q + 6) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 10
    point_index = (i * q + q + 7) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 11
    point_index = (i * q + q + 8) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])

```

```

for i in range(q):
    line_index = i * q + q + 12
    point_index = (i * q + q + 9) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 13
    point_index = (i * q + q + 10) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 14
    point_index = (i * q + q + 11) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 15
    point_index = (i * q + q + 12) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 16
    point_index = (i * q + q + 13) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 17
    point_index = (i * q + q + 14) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 18
    point_index = (i * q + q + 15) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 19
    point_index = (i * q + q + 16) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 20
    point_index = (i * q + q + 17) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 21

```

```

    point_index = (i * q + q + 18) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 22
    point_index = (i * q + q + 19) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 23
    point_index = (i * q + q + 20) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 24
    point_index = (i * q + q + 21) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 25
    point_index = (i * q + q + 22) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 26
    point_index = (i * q + q + 23) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 27
    point_index = (i * q + q + 24) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 28
    point_index = (i * q + q + 25) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 29
    point_index = (i * q + q + 26) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 30
    point_index = (i * q + q + 27) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1

```

```

    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 31
    point_index = (i * q + q + 28) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 32
    point_index = (i * q + q + 29) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 33
    point_index = (i * q + q + 30) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 34
    point_index = (i * q + q + 31) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 35
    point_index = (i * q + q + 32) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 36
    point_index = (i * q + q + 33) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 37
    point_index = (i * q + q + 34) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 38
    point_index = (i * q + q + 35) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 39
    point_index = (i * q + q + 36) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])

```

```

for i in range(q):
    line_index = i * q + q + 40
    point_index = (i * q + q + 37) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 41
    point_index = (i * q + q + 38) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 42
    point_index = (i * q + q + 39) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 43
    point_index = (i * q + q + 40) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 44
    point_index = (i * q + q + 41) % (q + 1) + 1
    incidence_matrix[line_index][point_index] = 1
    clauses.append([line_index, -point_index])
    clauses.append([-line_index, point_index])
for i in range(q):
    line_index = i * q + q + 45
    point_index = (i * q + q + 42) % (q + 1) + 1
# ... [truncated, full source in replay tarball]

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Critic challenges conjecture; support fraction ≥ 0.8 not unambiguously confirmed | next:
Verify actual support fraction from test metrics

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106346
2	propose	ollama_remote	qwen3:8b	0	0	79966
3	preregistration	ollama_remote	qwen3:8b	0	0	19743
4	novelty	ollama_remote	qwen3:8b	0	0	16359
5	novelty	ollama_remote	qwen3:8b	0	0	19918
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17984
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	83931
8	critic	ollama_remote	qwen3:8b	0	0	25163
9	judge	ollama_remote	qwen3:8b	0	0	12026

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 381435 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/595a1117d469.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/595a1117d469.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/595a1117d469.tar.gz` (if generated)